

Documentación Técnica — Proyecto E-Commerce Java

Proyecto académico orientado a arquitectura backend en Java

1. Objetivo del Proyecto

El proyecto fue desarrollado como una aplicación backend de consola en Java, orientada a objetos y diseñada bajo una arquitectura modular por capas. El objetivo principal fue construir un sistema de e-commerce simple que permitiera gestionar productos y pedidos, aplicando buenas prácticas de programación y conceptos fundamentales del desarrollo backend.

Durante el desarrollo se trabajó especialmente sobre:

- Programación Orientada a Objetos (POO)
- Encapsulamiento
- Modelado de entidades
- Lógica de negocio mediante servicios
- Validaciones reutilizables
- Manejo de excepciones personalizadas
- Organización profesional del código en paquetes

2. Arquitectura Utilizada

La aplicación utiliza una arquitectura por capas simplificada:

UI → Service → Model

Cada capa posee una responsabilidad específica:

- UI: interacción con el usuario mediante consola.
- Service: contiene la lógica de negocio y las operaciones principales.
- Model: representa las entidades del dominio.
- Exception: manejo de errores personalizados.
- Util: validaciones y funcionalidades auxiliares.

Esta separación mejora considerablemente la mantenibilidad, la legibilidad y la escalabilidad del proyecto.

3. Estructura del Proyecto

La estructura del proyecto fue organizada de la siguiente manera:

```
com.techlab.ecommerce
|
├── exception
├── model
├── service
├── ui
├── util
└── Main
```

Esta organización permite:

- Mejor lectura del proyecto
- Separación lógica de responsabilidades
- Mantenimiento más sencillo
- Mayor escalabilidad futura

4. Modelado de Entidades

Las entidades principales implementadas fueron:

- Producto
- Pedido
- LineaPedido

El diseño utiliza composición de manera correcta:

Pedido → List<LineaPedido> → Producto

Gracias a esta estructura, el sistema puede representar correctamente:

- Productos asociados a un pedido
- Cantidades solicitadas
- Cálculo de subtotales
- Cálculo del total general
- Control y actualización de stock

5. Lógica de Negocio

La lógica de negocio fue centralizada dentro de las clases Service, evitando colocar operaciones importantes directamente en la interfaz de usuario.

Entre las funcionalidades implementadas se encuentran:

- Alta de productos
- Búsqueda por ID
- Actualización de datos
- Eliminación de productos
- Validación de stock
- Reducción automática de stock
- Creación y gestión de pedidos

Este enfoque sigue una estructura similar a la utilizada en aplicaciones backend profesionales.

6. Manejo de Excepciones

Se implementaron excepciones personalizadas para mejorar el control de errores y la claridad del código.

Excepciones utilizadas:

- `ProductoNoEncontradoException`
- `StockInsuficienteException`

El uso de excepciones personalizadas permite:

- Mejor legibilidad
- Mayor mantenibilidad
- Manejo controlado de errores
- Mensajes más descriptivos
- Mejor experiencia de depuración

7. Validaciones

El proyecto incorpora una clase utilitaria de validaciones reutilizable.

Las validaciones implementadas permiten:

- Controlar datos inválidos
- Evitar estados inconsistentes
- Centralizar reglas comunes del sistema

A nivel profesional, estas validaciones podrían evolucionar hacia:

- Validaciones por dominio
- Bean Validation
- Uso de anotaciones como @NotNull, @Positive, @Size, entre otras

8. Buenas Prácticas Aplicadas

El proyecto demuestra múltiples buenas prácticas relacionadas con el desarrollo backend:

- Separación de responsabilidades
- Uso correcto de encapsulamiento
- Organización modular
- Utilización de servicios
- Excepciones personalizadas
- Métodos pequeños y legibles
- Nombres descriptivos
- Reutilización de lógica
- Código mantenible y escalable

9. Mejoras Profesionales Recomendadas

Para evolucionar el proyecto hacia un entorno más cercano a producción, se recomienda implementar:

- Reemplazo de double por BigDecimal
- Persistencia de datos con PostgreSQL o MySQL
- Implementación de capa Repository
- Uso de interfaces
- Testing unitario con JUnit
- Logging profesional
- Separación completa entre UI y lógica de negocio
- Migración a Spring Boot
- Desarrollo de API REST
- Dockerización del proyecto

10. Evolución Profesional del Proyecto

La evolución natural del proyecto podría seguir la siguiente progresión:

Aplicación de Consola

↓

Backend Modular

↓

REST API con Spring Boot

↓

Persistencia SQL

↓

Testing + Docker + CI/CD

↓

Arquitectura Profesional

El proyecto actual ya representa una base sólida para un perfil backend junior.

11. Conclusión

El proyecto demuestra una buena comprensión de los fundamentos del desarrollo backend y de la Programación Orientada a Objetos.

Los aspectos más destacados son:

- Organización clara del código
- Separación de responsabilidades
- Modelado correcto de entidades
- Arquitectura modular
- Uso de excepciones y validaciones
- Implementación de lógica de negocio estructurada

Aunque todavía existen mejoras posibles para un entorno de producción real, la base arquitectónica construida es correcta y representa un excelente punto de partida para evolucionar hacia aplicaciones backend profesionales.